

**APM_52448_EP - Deep Learning in Finance****Project 1 - Deep pricing and calibration of the Heston model**

Olivier Féron

PLEASE ENTER YOUR FULL NAMES HERE:

- TEJODONKENG MENKEM LUC NELSON
- MEMBER 2

DEADLINE: March 4, 2026 (5:00 pm)**Please send both pdf and ipynb files with name : Name1_Name2_Project1**

0. Setup

```

1 # --- Imports & global settings ---
2 import os, math
3 import numpy as np
4 import pandas as pd
5 import torch
6 import torch.nn as nn
7 from torch.utils.data import TensorDataset, DataLoader
8 from scipy.stats import norm
9 from scipy.optimize import minimize
10 from time import perf_counter
11 import datetime
12 import joblib
13 import matplotlib.pyplot as plt
14 from scipy.interpolate import griddata
15 # Training with early stopping on validation RMSE
16 os.makedirs("Data", exist_ok=True)
17 file_save_best_model = "Data/best_mlp.pt"
18 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
19 SEED=0
20 np.random.seed(SEED)
21 torch.manual_seed(SEED)

```

<torch._C.Generator at 0x7f9476764cd0>

1 The Heston stochastic volatility model

The Heston Model (Heston, 1993) under the risk-neutral measure $\mathbb{Q}$:

$$\begin{aligned}\frac{dS_t}{S_t} &= r dt + \sqrt{v_t} dW_t, \\ dv_t &= \alpha(\beta - v_t) dt + \delta\sqrt{v_t} dB_t, \\ d\langle W, B \rangle_t &= \rho dt.\end{aligned}$$

We calibrate the parameter vector $\Theta = (v_0, \alpha, \beta, \delta, \rho)$.

The Heston model is well adapted to fit the volatility smile: Heston model's parameters can adapt lots of smile shapes.

We consider **European call options** with payoff $(S_T - K)^+$.

In this project, calibration is performed **on call prices** (as in your Chapter 3 tutorials), not on implied volatilities.

You will use a neural network surrogate fast pricer:

$$(\Theta, K) \mapsto C_{\text{Heston}}(K, T)$$

for a **single maturity** T (same setting as your market dataset).

Your task

Inspired by your Chapter 3 notebooks (FastPricing_BS_Surrogate and Calibration_NN):

1. Generate simulations of the Heston model
2. Implement the Heston pricer (via Monte Carlo) to generate synthetic training data
3. Train a deep fast pricer on the synthetic data and check performance
4. Train a deep fast pricer
5. Analyze the results and find out what was wrong in the procedure

2 Monte Carlo simulation under the Heston model

In this project, we simulate asset price trajectories under the Heston stochastic volatility model in order to compute option prices via Monte Carlo.

2.2 Time discretization

We discretize time:

$$t_n = n\Delta t, \quad \Delta t = \frac{T}{N}.$$

We consider correlated Brownian increments:

$$\begin{aligned} \Delta W_n &= \sqrt{\Delta t} Z_1^n, \\ \Delta B_n &= \sqrt{\Delta t} \left(\rho Z_1^n + \sqrt{1 - \rho^2} Z_2^n \right). \end{aligned}$$

with $Z_1^n, Z_2^n \sim \mathcal{N}(0, 1)$

2.3 Variance discretization

A simple Euler scheme gives:

$$v_{n+1} = v_n + \alpha(\beta - v_n)\Delta t + \delta\sqrt{v_n} \Delta B_n.$$

However, this may produce negative values. A common practical fix is the **Full Truncation Euler scheme**:

$$v_{n+1} = v_n + \alpha(\beta - v_n^+)\Delta t + \delta\sqrt{v_n^+} \Delta B_n,$$

with $v_n^+ = \max(v_n, 0)$.

2.4 Stock price discretization

Using a log-Euler scheme:

$$S_{n+1} = S_n \exp\left((r - \frac{1}{2}v_n^+)\Delta t + \sqrt{v_n^+} \Delta W_n\right).$$

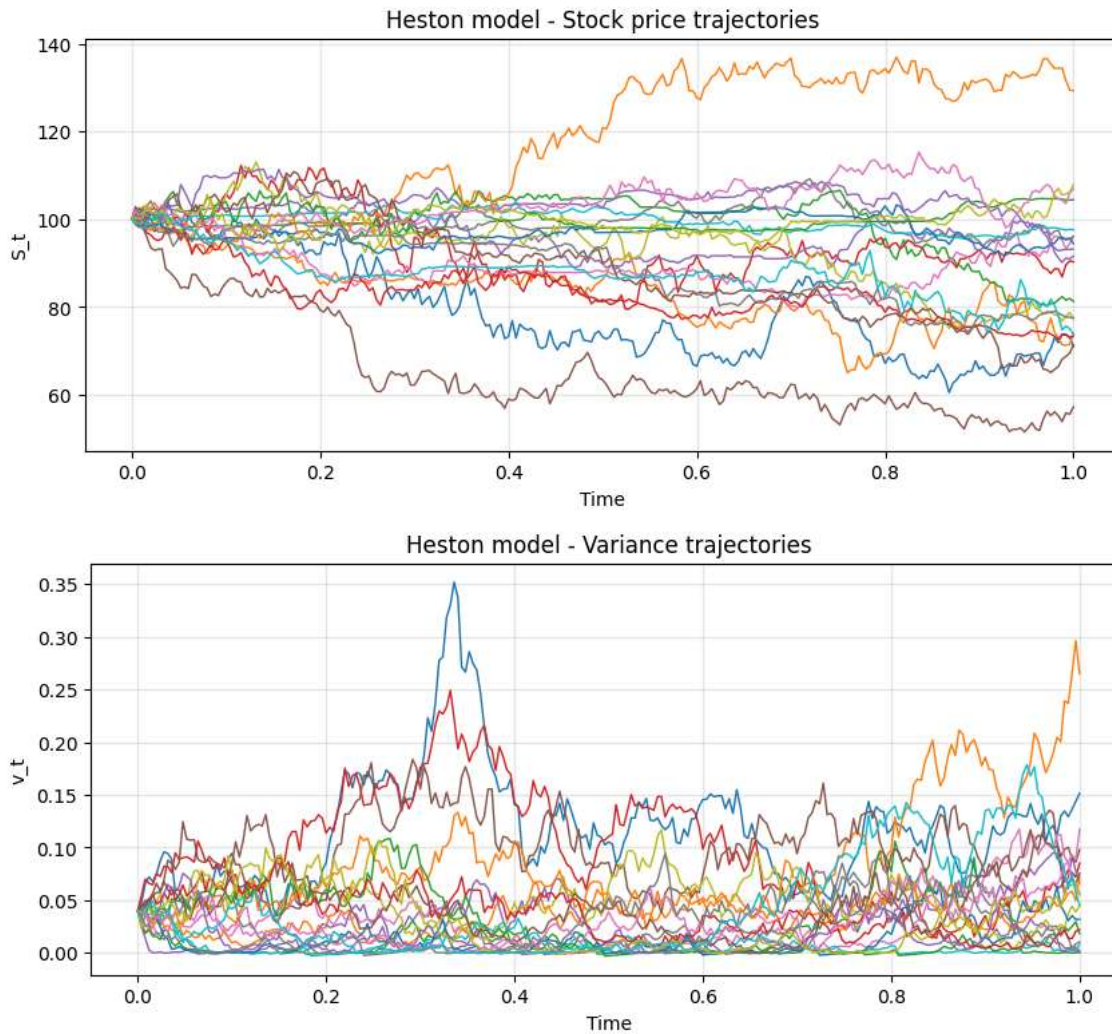
2.1 Implement a function that generates trajectories from the Heston model.

```
1 def simulate_heston(S0, v0, r, alpha, beta, delta, rho, T, n_steps, n_paths):
2     """
3     Simulate trajectories under the Heston model:
4         dS_t / S_t = r dt + sqrt(v_t) dW_t
5         dv_t = alpha (beta - v_t) dt + delta sqrt(v_t) dB_t
6         d<W,B>_t = rho dt
7     using Full Truncation Euler scheme.
8
9     Outputs
10    -----
11    S : array (n_paths, n_steps+1)
12    v : array (n_paths, n_steps+1)
13    """
14
15    dt = T / n_steps
16
17    S_paths = np.zeros((n_paths, n_steps + 1))
18    v_paths = np.zeros((n_paths, n_steps + 1))
19
20    S_paths[:, 0] = S0
```

```

21     v_paths[:, 0] = v0
22
23     for t in range(n_steps):
24         S_paths[:, t + 1] = S_paths[:, t] * np.exp(
25             (r - 0.5 * np.maximum(v_paths[:, t], 0)) * dt + np.sqrt(np.maximum(v_paths[:, t], 0)) * np.random.normal(size=n
26             v_paths[:, t + 1] = v_paths[:, t] + alpha * (beta - np.maximum(v_paths[:, t], 0)) * dt + delta * np.sqrt(np.maximum
27
28     return S_paths, v_paths
29
30
31
32 # Parameters
33 S0 = 100
34 v0 = 0.04
35 r = 0.02
36 alpha = 2.0
37 beta = 0.04
38 delta = 0.6
39 rho = -0.7
40 T = 1.0
41
42 n_steps = 250
43 n_paths = 20
44
45 # Simulate
46 S_paths, v_paths = simulate_heston(
47     S0, v0, r,
48     alpha, beta, delta, rho,
49     T, n_steps, n_paths,
50 )
51
52 # Time grid
53 t_grid = np.linspace(0, T, n_steps + 1)
54
55 # Plot stock paths
56 plt.figure(figsize=(10,4))
57 for i in range(n_paths):
58     plt.plot(t_grid, S_paths[i], lw=1)
59 plt.title("Heston model - Stock price trajectories")
60 plt.xlabel("Time")
61 plt.ylabel("S_t")
62 plt.grid(alpha=0.3)
63 plt.show()
64
65 # Plot variance paths
66 plt.figure(figsize=(10,4))
67 for i in range(n_paths):
68     plt.plot(t_grid, v_paths[i], lw=1)
69 plt.title("Heston model - Variance trajectories")
70 plt.xlabel("Time")
71 plt.ylabel("v_t")
72 plt.grid(alpha=0.3)
73 plt.show()

```



3. Call option price and dataset generation

Now it is possible to approximate call option prices by Monte Carlo

3.1 Implement a function that generates trajectories from the Heston model.

```
1 def heston_call_price_mc(S0, v0, r, alpha, beta, delta, rho, T, K, n_steps, n_paths):
2     return np.exp(-r * T) * np.mean(np.maximum(simulate_heston(S0, v0, r, alpha, beta, delta, rho, T, n_steps, n_paths)[0][:
```

3.1 Dataset generation

The objective is to randomly sample Heston parameters in the training domain $\mathcal{D}_{\text{domain}}$:

- $v_0 \in [0.01, 0.12]$
- $\alpha \in [0.3, 4.0]$
- $\beta \in [0.01, 0.12]$
- $\delta \in [0.1, 1.2]$
- $\rho \in [-0.95, -0.05]$
- $S_0 = 274.73$
- $K \in [60, 140]$
- $T = 0.2$

For each $x \in \mathcal{D}_{\text{domain}}$, we then compute Monte Carlo prices y using the Heston simulation scheme implemented previously.

The surrogate model will therefore learn the mapping:

$$(v_0, \alpha, \beta, \delta, \rho, S_0, K, T) \mapsto C_{\text{Heston}}(S_0, K, T).$$

3.1 Implement a function `build_dataset(n_samples, n_simulations)` that builds a dataset $(x_i, y_i) \in \mathcal{D}_{\text{domain}}$, for $i = 1, \dots, n_{\text{samples}}$

3.2 Use this function to generate a dataset of 10,000 samples, and split the dataset into $X_{\text{train}}, y_{\text{train}}, X_{\text{val}}, y_{\text{val}}, X_{\text{test}}, y_{\text{test}}$ with a respective repartition (50%, 10%, 40%).

```

1 # -----
2 # Dataset builder
3 # -----
4
5
6
7 def build_dataset(n_samples, n_paths, n_steps=100, verbose=True):
8
9     def sample_domain(n_samples):
10         S0 = [274.73] * n_samples
11         v0 = np.random.uniform(0.01, 0.12, n_samples)
12         r=np.ones(n_samples) * 0.02 # vu que r reste fixe à 0.02
13         alpha = np.random.uniform(0.3, 4.0, n_samples)
14         beta = np.random.uniform(0.01, 0.12, n_samples)
15         delta = np.random.uniform(0.1, 1.2, n_samples)
16         rho = np.random.uniform(-0.95, -0.05, n_samples)
17         T = [0.2] * n_samples
18         K = np.random.uniform(60, 140, n_samples)
19         X=np.stack((S0, v0, r,alpha, beta, delta, rho, T, K), axis=1)
20
21         return X
22     t0 = perf_counter()
23     X = sample_domain(n_samples)
24     y = np.zeros((n_samples, 1))
25
26     for i in range(n_samples):
27         y[i] = heston_call_price_mc(*X[i], n_steps=n_steps, n_paths=n_paths)
28         if verbose and (i+1) % max(1, n_samples//10) == 0:
29             print(f"{i+1}/{n_samples} done")
30     t1 = perf_counter()
31     if verbose:
32         print(f"Dataset generated in {t1-t0:.2f}s (n_samples={n_samples}, n_simulations={n_paths})")
33     return X, y.reshape(-1,1)
34
35 # Generate dataset
36 X, y = build_dataset(n_samples=10000,n_paths=5000,n_steps=50)
37
38 # -----
39 # Train / Val / Test split
40 # -----
41
42 def random_split(X, y, train_ratio=0.5, val_ratio=0.1):
43
44     # Lengths
45     n = X.shape[0]
46     ny = y.shape[0]
47     # Check of equal number of samples
48     if n != ny:
49         raise ValueError(f"X and y must have the same number of samples (X:{n}, y:{ny})")
50
51     n_train = int(n * train_ratio)
52     n_val = int(n * val_ratio)
53
54     # Random permutation of the samples
55     idx = np.random.permutation(n)
56
57     # Split indices
58     idx_train = idx[:n_train]
59     idx_val = idx[n_train:n_train + n_val]
60     idx_test = idx[n_train + n_val:]
61
62
63     # Split Data
64     X_train, y_train = X[idx_train], y[idx_train]
65     X_val, y_val = X[idx_val], y[idx_val]
66     X_test, y_test = X[idx_test], y[idx_test]
67
68     return (X_train, y_train), (X_val, y_val), (X_test, y_test)
69
70

```

```

71
72 (X_train, y_train), (X_val, y_val), (X_test, y_test) = random_split(X, y)
73
74 print("Train:", X_train.shape)
75 print("Val  :", X_val.shape)
76 print("Test :", X_test.shape)

```

```

1000/10000 done
2000/10000 done
3000/10000 done
4000/10000 done
5000/10000 done
6000/10000 done
7000/10000 done
8000/10000 done
9000/10000 done
10000/10000 done
Dataset generated in 207.55s (n_samples=10000, n_simulations=5000)
Train: (5000, 9)
Val  : (1000, 9)
Test : (4000, 9)

```

3.2 Data normalization

3.3 Standardize all the data in an appropriate way

```

1 # YOUR CODE HERE
2
3 from sklearn.preprocessing import StandardScaler
4 scaler_X = StandardScaler().fit(X_train)
5 X_train_s = scaler_X.transform(X_train)
6 X_val_s   = scaler_X.transform(X_val)
7 X_test_s  = scaler_X.transform(X_test)
8
9 scaler_y = StandardScaler().fit(y_train)
10 y_train_s = scaler_y.transform(y_train)
11 y_val_s   = scaler_y.transform(y_val)
12 y_test_s  = scaler_y.transform(y_test)

```

```

1 # Transform Dataset in Torch objects
2 train_ds = TensorDataset(torch.tensor(X_train_s, dtype=torch.float32),
3                             torch.tensor(y_train_s, dtype=torch.float32))
4 val_ds   = TensorDataset(torch.tensor(X_val_s, dtype=torch.float32),
5                             torch.tensor(y_val_s, dtype=torch.float32))
6 test_ds  = TensorDataset(torch.tensor(X_test_s, dtype=torch.float32),
7                             torch.tensor(y_test_s, dtype=torch.float32))
8
9 train_loader = DataLoader(train_ds, batch_size=64, shuffle=False) # Shuffle => mix or not the data. False => keep time ord
10 val_loader  = DataLoader(val_ds, batch_size=64, shuffle=False)
11 test_loader = DataLoader(test_ds, batch_size=64, shuffle=False)
12
13 # Sanity check
14 print("X_train mean (approx):", X_train_s.mean(axis=0))
15 print("X_train std (approx):", X_train_s.std(axis=0))
16
17 # Saving scaling parameters
18 joblib.dump(scaler_X, "scaler_X.joblib")
19 joblib.dump(scaler_y, "scaler_y.joblib")

```

```

X_train mean (approx): [ 1.42108547e-11  3.20876659e-16  5.51642065e-16 -5.12119236e-15
-4.61642391e-15  9.14690546e-16  8.58784155e-15 -1.80688797e-14
-5.01709785e-16]
X_train std (approx): [0.  1.  0.  1.  1.  1.  0.  1.]
['scaler_y.joblib']

```

4. Surrogate model: a simple MLP

Now that all is ready to train a Neural Network, we consider a simple MLP surrogate model.

We propose a fully-connected network:

- **input dim:** 5 (features $(v_0, \alpha, \beta, \delta, \rho, S_0, K, T)$ standardized)
- **Hidden layers:** 3 layers with sizes 128 --> 128 --> 64, all with ReLU

- **Output dim:** 1 (scaled (standardized) price)
- **Loss function:** MSE (nn.MSELoss)
- **Optimizer:** Adam (Learning rate $1e^{-3}$)

4.1 Build the class MLP with the above characteristics

4.2 Implement the function `run_epoch(loader, train)` that runs one epoch

- sets `model.train()` if `train=True` otherwise `model.eval()`
- iterates over the loader, sends
- returns the losses

4.3 Implement the training procedure on 250 Epochs, with an overfitting control

- with a patience counter of 20 (if no loss decrease during 20 Epochs, then stop)
- save the best model in `Data/best_mlp.pt`

4.4 Plot the MSE losses (Learning Curves) for the train and val sets

```

1 # YOUR CODE HERE
2
3 # Training with early stopping on validation RMSE
4 import math
5 best_val = math.inf
6 patience= 20
7 EPOCHS = 250
8 file_save_best_model = "fast_pricing_mlp.pt"
9 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
10
11 # Model
12 class MLP(nn.Module):
13     def __init__(self, in_dim):
14         super().__init__()
15         self.net = nn.Sequential(
16             nn.Linear(in_dim, 128),
17             nn.ReLU(),
18             nn.Linear(128, 128),
19             nn.ReLU(),
20             nn.Linear(128, 64),
21             nn.ReLU(),
22             nn.Linear(64,1)
23         )
24     def forward(self, x):
25         return self.net(x)
26
27
28 model = MLP(in_dim=X_train_s.shape[1]).to(device)
29
30 criterion = nn.MSELoss()
31 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
32
33
34 def run_epoch(loader, train=False):
35     if train: model.train()
36     else: model.eval()
37     losses = []
38     for xb, yb in loader:
39         xb, yb = xb.to(device), yb.to(device)
40         pred = model(xb)
41         loss = criterion(pred, yb)
42         if train:
43             optimizer.zero_grad(set_to_none=True)
44             loss.backward()
45             optimizer.step()
46         losses.append(loss.item())
47     return np.mean(losses)
48
49
50 patience_cnt = 0
51 train_data, val_data = [], []
52 for epoch in range(1, EPOCHS+1):
53     tr = run_epoch(train_loader, train=True) # Can mix the order since we are not working with time series
54     va = run_epoch(val_loader, train=False) # Must keep the order to match with predicted value
55     train_data.append(tr); val_data.append(va)
56     val_mse = va
57     if val_mse < best_val - 1e-6:

```

```

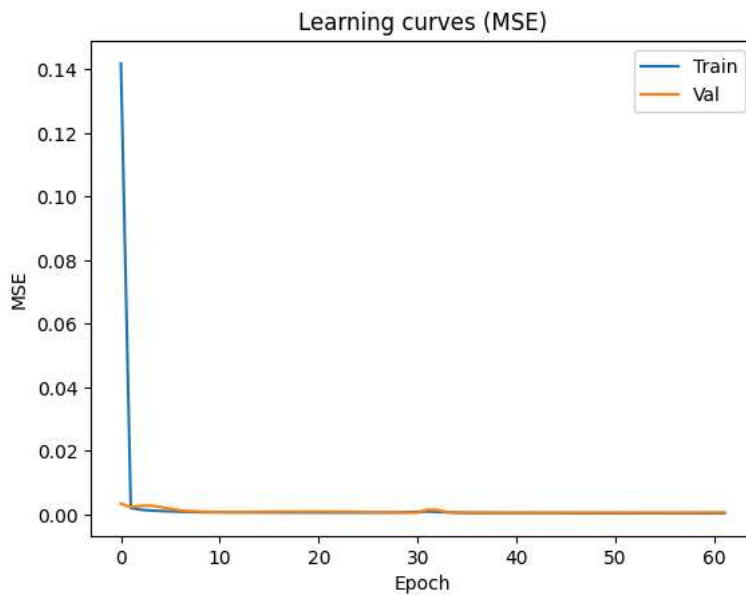
58     best_val = val_mse
59     patience_cnt = 0
60     torch.save(model.state_dict(), file_save_best_model)
61     else:
62         patience_cnt += 1
63     if epoch % 20 == 0:
64         print(f"Epoch {epoch:03d} | Train MSE={tr:.6f} | Val MSE={val_mse:.6f}")
65     if patience_cnt >= patience:
66         print("Early stopping.")
67         break
68
69 plt.figure()
70 plt.plot(train_data)
71 plt.plot(val_data)
72 plt.title("Learning curves (MSE)")
73 plt.xlabel("Epoch")
74 plt.ylabel("MSE")
75 plt.legend(["Train", "Val"])
76 plt.show()

```

```

Epoch 020 | Train MSE=0.000658 | Val MSE=0.000925
Epoch 040 | Train MSE=0.000492 | Val MSE=0.000612
Epoch 060 | Train MSE=0.000444 | Val MSE=0.000654
Early stopping.

```



5. Evaluation (accuracy)

Now the surrogate model is trained, let's evaluate its performance on the test set

5.1 Load the best model saved in `file_save_best_model` and the scaling parameters with command `joblib.load` and evaluate with `model.eval()`

5.2 Compute the predicted `y_test_pred` from `X_test` (test_loader tensor) and unscale to get `y_test_hat`

5.3 Compute the MAE and RMSE, and give the scatter plot MC price vs. NN price

5.4 Give the scatter plot BS price vs. NN price

```

1 def Black_Scholes_Formula(vol, risk_free_rate, current_price, strike, maturity):
2     r"""
3     Black_Scholes_Formula(vol, risk_free_rate, current_price, strike, maturity)
4     Function computing the Black-Scholes formula to obtain the price of a Call option
5
6     Inputs
7     -----
8     vol          : volatility of the underlying
9     risk_free_rate : value of the risk free rate
10    current_price  : current value of the underlying price (X_t)
11    strike         : value of the strike (K)
12    maturity       : Maturity of the option (T-t)
13

```

```

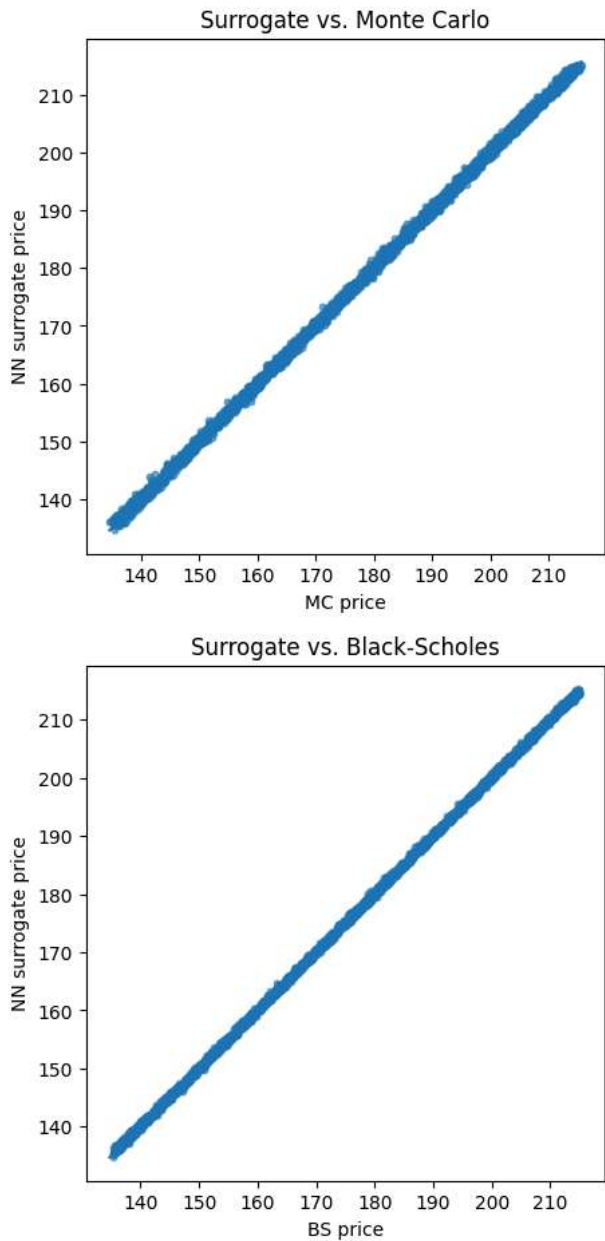
14 Output
15 -----
16 Value      : Call option price from BS formula :  $X_0 \Phi(d_1) - K e^{-r(T-t)} \Phi(d_2)$ 
17 ""
18 # Computation of d_1 and d_2 of the Black-Scholes Formula
19 d_1 = (np.log(current_price/strike) + (risk_free_rate + 0.5 * vol**2)*maturity)/(vol * np.sqrt(maturity))
20 d_2 = d_1 - vol * np.sqrt(maturity)
21 # Black-Scholes formula
22 Value = current_price * norm.cdf(d_1) - strike * np.exp(- risk_free_rate * maturity) * norm.cdf(d_2)
23 return Value
24
25

```

```

1 # YOUR CODE HERE
2 # Load the best model (saved during learning)
3 model.load_state_dict(torch.load(file_save_best_model, map_location=device))
4 model.eval()
5
6 # Also import the scaling parameters
7 scaler_X = joblib.load("scaler_X.joblib")
8 scaler_y = joblib.load("scaler_y.joblib")
9
10 # Prediction on the test set
11 with torch.no_grad():
12     y_test_pred = []
13     for xb, _ in test_loader:
14         xb = xb.to(device)
15         y_test_pred.append(model(xb).cpu().numpy())
16     y_test_pred = np.concatenate(y_test_pred, axis=0)
17
18 # Unscale
19 y_test_hat = scaler_y.inverse_transform(y_test_pred)
20 # MAE and RMSE
21
22 # Scatterplot MC vs. NN
23 plt.figure(figsize=(5,5))
24 plt.scatter(y_test, y_test_hat, s=10, alpha=0.6)
25 m = float(min(y_test.min(), y_test_hat.min()))
26 M = float(max(y_test.max(), y_test_hat.max()))
27 plt.plot([m, M], [m, M])
28 plt.xlabel("MC price")
29 plt.ylabel("NN surrogate price")
30 plt.title("Surrogate vs. Monte Carlo")
31 plt.show()
32
33
34 #####
35 # BS vs. NN
36 # Compute Black-Scholes prices
37 sigma = X_test[:,1]**0.5
38 r = X_test[:,2]
39 S0 = X_test[:,0]
40 K = X_test[:,8]
41 Tmt = X_test[:,7]
42
43 y_bs = Black_Scholes_Formula(sigma, r, S0, K, Tmt)
44 # Scatterplot MC vs. NN
45 plt.figure(figsize=(5,5))
46 plt.scatter(y_bs, y_test_hat, s=10, alpha=0.6)
47 m = float(min(y_bs.min(), y_test_hat.min()))
48 M = float(max(y_bs.max(), y_test_hat.max()))
49 plt.plot([m, M], [m, M])
50 plt.xlabel("BS price")
51 plt.ylabel("NN surrogate price")
52 plt.title("Surrogate vs. Black-Scholes")
53 plt.show()
54

```



On voit que les prix donnés par le modèle surrogate basés sur le modèle de Heston sont très similaires à ceux qu'on obtiendrait en faisant un monte carlo. Ce qui justifie que cela constitue une benchmark assez rapide dans la pratique.

De plus, ces prix sont assez similaires à ceux obtenus par le modèle de Black scholes, ce qui confirme que le modèle de black scholes est une excellente approximation

✓ 6 Calibration on real data

The objective is to calibrate the Black-Scholes model, i.e. to estimate the implied volatility σ_{imp} .

We propose to estimate $\theta = (v_0, \alpha, \beta, \delta, \rho)$ by minimizing the quadratic error:

$$\sigma_{\text{imp}} = \arg \min_{\theta} J^{\text{method}}(\theta)$$

with

$$J^{\text{method}}(\theta) = \frac{1}{I} \sum_{i=1}^I \|C_i^{\text{mkt}} - C_i^{\text{NN}}(\theta)\|^2$$

where $C_i^{\text{NN}}(\sigma)$ is the price obtained from the NN Surrogate fast pricer.

✓ 6.1 First step : download and prepare the data

[illegible]

	Contract Name	Strike	Last Price	Bid	Ask	Change	Volume	Open Interest
0	AAPL261218C00005000	5	264.64	267.50	271.30	0.0	1.0	2.0
1	AAPL261218C00010000	10	245.23	251.00	254.85	0.0	10.0	15.0
2	AAPL261218C00015000	15	186.06	238.95	242.35	0.0	4.0	5.0
3	AAPL261218C00020000	20	240.80	255.65	259.55	0.0	1.0	5.0
4	AAPL261218C00025000	25	184.65	229.35	232.75	0.0	1.0	2.0
...	...	...	...	...	...	...	...	...
80	AAPL261218C00490000	490	0.51	0.43	0.47	0.0	1.0	42.0
81	AAPL261218C00500000	500	0.40	0.36	0.40	0.0	6.0	223.0
82	AAPL261218C00510000	510	0.40	0.31	0.35	0.0	5.0	26.0
83	AAPL261218C00520000	520	0.30	0.27	0.31	0.0	100.0	305.0
84	AAPL261218C00530000	530	0.26	0.23	0.27	0.0	NaN	NaN

```
1 #definir X_mkt
2 X_mkt = np.stack((np.full_like(K_vec, S0), # S0
3                   np.full_like(K_vec, 0.01), # v0
4                   np.full_like(K_vec, r), # r
5                   np.full_like(K_vec, 2.0), # alpha
6                   np.full_like(K_vec, 0.04), # beta
7                   np.full_like(K_vec, 0.6), # delta
8                   np.full_like(K_vec, -0.7), # rho
```

```

9         T_vec,          # T
10        K_vec          # K
11    ), axis=1)

```

6.2 Cost function

6.1 Build in the code below the function `nn_price_call_heston` that computes the price from the NN Surrogate Fast pricer.

```

1 # Global variables expected by loss_nn_heston
2 K_mkt = K_vec.copy()
3 T_mkt = T_vec.copy()
4 C_mkt = C_mkt_vec.copy()

```

```

1 def build_features_heston(v0, alpha, beta, delta, rho, S0, K, T):
2     '''
3     Creates the NN input features for the Heston surrogate model.
4
5     All inputs must be broadcastable arrays of same length.
6     '''
7     v0 = np.asarray(v0, dtype=float)
8     alpha = np.asarray(alpha, dtype=float)
9     beta = np.asarray(beta, dtype=float)
10    delta = np.asarray(delta, dtype=float)
11    rho = np.asarray(rho, dtype=float)
12    S0 = np.asarray(S0, dtype=float)
13    K = np.asarray(K, dtype=float)
14    T = np.asarray(T, dtype=float)
15
16    # Broadcast all arrays to the same shape
17    v0, alpha, beta, delta, rho, S0, K, T = np.broadcast_arrays(
18        v0, alpha, beta, delta, rho, S0, K, T
19    )
20
21    # Constant rate with same shape
22    r = np.full_like(v0, 0.01, dtype=float)
23
24    # Ensure all arrays have the same shape
25    assert v0.shape == alpha.shape == beta.shape == delta.shape == rho.shape == S0.shape == K.shape == T.shape, \
26        f"Shape mismatch: v0={v0.shape}, alpha={alpha.shape}, beta={beta.shape}, delta={delta.shape}, rho={rho.shape}, S0={
27
28    return np.stack([S0, v0, r, alpha, beta, delta, rho, T, K], axis=1)

```

```

1
2

```

```

1 def build_features_heston(v0, alpha, beta, delta, rho, S0, K, T):
2     '''
3     Creates the NN input features for the Heston surrogate model.
4
5     All inputs must be broadcastable arrays of same length.
6     '''
7     v0 = np.asarray(v0, dtype=float)
8     alpha = np.asarray(alpha, dtype=float)
9     beta = np.asarray(beta, dtype=float)
10    delta = np.asarray(delta, dtype=float)
11    rho = np.asarray(rho, dtype=float)
12    S0 = np.asarray(S0, dtype=float)
13    K = np.asarray(K, dtype=float)
14    T = np.asarray(T, dtype=float)
15
16    # Broadcast all arrays to the same shape
17    v0, alpha, beta, delta, rho, S0, K, T = np.broadcast_arrays(
18        v0, alpha, beta, delta, rho, S0, K, T
19    )
20
21    # Constant rate with same shape
22    r = np.full_like(v0, 0.01, dtype=float)
23
24    # Ensure all arrays have the same shape
25    assert v0.shape == alpha.shape == beta.shape == delta.shape == rho.shape == S0.shape == K.shape == T.shape, \
26        f"Shape mismatch: v0={v0.shape}, alpha={alpha.shape}, beta={beta.shape}, delta={delta.shape}, rho={rho.shape}, S0={
27
28    return np.column_stack([S0, v0, r, alpha, beta, delta, rho, T, K])

```

```

1
2
3 def nn_price_call_heston(model, scaler_X, scaler_y, X):
4     '''
5     Uses the trained surrogate model to predict call prices for given Heston parameters and option specs.
6
7     Inputs
8     -----
9     model : trained surrogate model (PyTorch)
10    scaler_X : scaler used to scale the input features during training (sklearn StandardScaler)
11    scaler_y : scaler used to scale the target during training (sklearn StandardScaler)
12    X : array of shape (n_samples, 8) containing the input features [v0, alpha, beta, delta, rho, S0, K, T]
13
14    Output
15    -----
16    C : array of shape (n_samples,) containing the predicted call prices
17    '''
18    # Scale input features
19    X_s = scaler_X.transform(X)
20
21    # Predict scaled price with the surrogate model
22    with torch.no_grad():
23        X_tensor = torch.tensor(X_s, dtype=torch.float32).to(next(model.parameters()).device)
24        y_pred_s = model(X_tensor).cpu().numpy()
25
26    # Unscale predicted price
27    C = scaler_y.inverse_transform(y_pred_s)
28
29    return C.reshape(-1)

```

```

1
2
3
4 def loss_nn_heston(theta):
5     '''
6     theta = [v0, alpha, beta, delta, rho]
7     '''
8     v0, alpha, beta, delta, rho = theta
9
10    # Génération des caractéristiques
11    X = build_features_heston(
12        v0 * np.ones_like(K_mkt),
13        alpha * np.ones_like(K_mkt),
14        beta * np.ones_like(K_mkt),
15        delta * np.ones_like(K_mkt),
16        rho * np.ones_like(K_mkt),
17        S0 * np.ones_like(K_mkt),

```

```

18     K_mkt,
19     T_mkt
20 )
21
22
23 # Calcul des prix avec le modèle
24 C = nn_price_call_heston(model, scaler_X, scaler_y, X).reshape(-1)
25
26 # Vérification des dimensions
27 assert C.shape == C_mkt.shape, f"Dimensions incompatibles : C={C.shape}, C_mkt={C_mkt.shape}"
28
29 # Calcul de la perte
30 return float(np.mean((C - C_mkt) ** 2))
31
32 # Initial guess
33 theta_0 = np.array([0.04, 1.0, 0.04, 0.5, -0.5])
34
35 # Bounds
36 HES_BOUNDS = [
37     (0.001, 0.2),      # v0
38     (0.1, 5.0),        # alpha
39     (0.001, 0.2),      # beta
40     (0.05, 2.0),       # delta
41     (-0.999, 0.999)    # rho
42 ]
43
44 # Vérification des dimensions des données
45 assert K_mkt.shape == T_mkt.shape == C_mkt.shape, "Les dimensions de K_mkt, T_mkt et C_mkt doivent correspondre."
46
47
48
49 # Minimisation
50 res = minimize(loss_nn_heston, theta_0, bounds=HES_BOUNDS, method="L-BFGS-B")
51
52 theta_star = res.x
53 print("Calibrated parameters:")
54 print("v0    =", theta_star[0])
55 print("alpha =", theta_star[1])
56 print("beta  =", theta_star[2])
57 print("delta =", theta_star[3])
58 print("rho   =", theta_star[4])
59 print("Final loss =", res.fun)
60
61 C_nn = np.zeros_like(K_vec, dtype=float)
62
63 for i, (Ki, Ti) in enumerate(zip(K_vec, T_vec)):
64
65     X_i = build_features_heston(
66         theta_star[0], theta_star[1], theta_star[2], theta_star[3], theta_star[4],
67         S0, Ki, Ti
68     )
69
70     C_nn[i] = nn_price_call_heston(model, scaler_X, scaler_y, X_i)
71
72
73
74
75 plt.figure(figsize=(8, 4.5))
76
77 plt.plot(K_vec, C_nn, 'o-', color='b', lw=1,
78         label='NN price (Heston calibrated)')
79 plt.plot(K_vec, C_mkt_vec, 'x-', color='r', lw=1,
80         label='Market price')
81
82 plt.xlabel('Strike K')
83 plt.ylabel('Call price')
84 plt.grid(True, alpha=0.3)
85 plt.legend()
86 plt.tight_layout()
87 plt.show()

```

